

Drone Garage

2/13/2026

To-Do

- **PRIMARY:** Focus on ESP mesh network metadata collection
 - Use wireshark for better monitoring
 - Collect everything that has to do with the network that the wireshark sniffs
 - Measure what happens when multiple ESPs are communicating
- Afterwards...
 - Get GPS data and send to the rest of the mesh
 - GPS data from multiple drones at the same time

ESP Mesh Data Collection

- Theorcrafting:
 - **Stevens IOT**
 - Wireshark on our laptops while ESPs on Stevens IOT? But they're not, the PIs are and Painlessmesh has them on their own network tho no? Even if we could they'd be encrypted ig?
 - **On the PIs**
 - This possible? Like wireshark on the pi monitoring the esp through the serial connection? Prob not?
 - Found an esp8266 pcap serial streaming live to a wireshark:
https://github.com/z4ziggy/esp8266_pcap_serial
 - Make our own? (using <https://github.com/spacehuhn/ArduinoPcap>)

Throughput Measurements - Dom

- Created a script on the PI (uart_flood.py) that sends about 5750 B/s over the uart (PI -> ESP) So UART -> ESP then ESP broadcasts the message
- ESP is using broadcast messaging
- Set up tasks in the mesh code to measure the tx and rx
- We get about 50 kbps tx and 25kbps rx (not terrible for painless mesh i think?)
 - **Play with the task execution (sendSerialThroughMesh) time to get them to match better (higher number between task execution means they will match better)**
- See next slide for images - dont mind the fall off in the tx image I was stopping the flood script

Throughput Measurements Explained - Dom

This is application level throughput not the wifi rate.

- TX is how fast the node is injecting messages into the mesh
- RX is how fast the other node is successfully receiving and decoding messages
- TX is calculated: for every mesh packet sent increase the packets sent var, increase the tx_wire_bytes var by the size of the JSON send over mesh and increase the tx_payload var for the real UART payload before hex encoding
 - $\text{Msg per second} = \text{tx_msgs} / \text{dt}$, $\text{wire bytes per second} = \text{tx_wire_bytes} / \text{dt}$ and $\text{payload bytes per second} = \text{tx_payload} / \text{dt}$
- RX is calculated: for every mesh packet received increase the rx_msg by one, increase the rx_wire_bytes by the msg length and increase the rx_payload_bytes by the decoded MAVlink bytes
 - $\text{elapsed time} = \text{dt} = \text{now} - \text{lastRateMs}$, $\text{msg per sec} = \text{rx_msgs} / \text{dt}$, $\text{bytes per sec} = \text{rx_wire_bytes} / \text{dt}$, $\text{kbps} = (\text{rx_wire_bytes} * 8 / \text{dt}) / 1000$, $\text{useful bytes per sec} = \text{rx_payload_bytes} / \text{dt}$

Dom - RX Second Node

```
@@{"type":"mesh_meta","event":"heartbeat","nodeName":"node2","meshId":2,"leaderId":2,"uptime":47276}
[tp] RX: 10.0 msg/s, 3060.0 B/s (24.48 kbps), payload 1280.0 B/s
[tp] TX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[node2] rx from 989188951: 306 bytes
[rx] type=m len=306
    MAVLink frame 128 bytes
[tp] RX: 10.0 msg/s, 3069.2 B/s (24.55 kbps), payload 1283.9 B/s
```

TX First Node

```
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tp] RX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[tp] TX: 21.0 msg/s, 6392.2 B/s (51.14 kbps), payload 2682.6 B/s
[node1] myMeshId=1 leader=1 me? no lastHB=1277 now=3282
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 128 bytes
[node1] myMeshId=1 leader=1 me? no lastHB=1277 now=4443
[tx] UART->mesh sent 128 bytes
[tp] RX: 0.0 msg/s, 0.0 B/s (0.00 kbps), payload 0.0 B/s
[tp] TX: 5.1 msg/s, 1568.1 B/s (12.54 kbps), payload 658.1 B/s
[tx] UART->mesh sent 128 bytes
[tx] UART->mesh sent 112 bytes
[tx] UART->mesh sent 21 bytes
[tx] UART->mesh sent 17 bytes
[tx] UART->mesh sent 51 bytes
[tx] UART->mesh sent 26 bytes
[tx] UART->mesh sent 11 bytes
[tx] UART->mesh sent 15 bytes
[tx] UART->mesh sent 28 bytes
[tx] UART->mesh sent 33 bytes
[tx] UART->mesh sent 17 bytes
[tx] UART->mesh sent 16 bytes
[tx] UART->mesh sent 16 bytes
[tx] UART->mesh sent 13 bytes
[tx] UART->mesh sent 17 bytes
[tx] UART->mesh sent 25 bytes
```

Wireshark

- Connect to ESP network before starting
- Monitor on WiFi

No.	Time	Source	Destination	Protocol	Length	Info
961	30.147968	10.195.10.100	10.195.10.1	TCP	66	[TCP Retransmission]
962	30.147984	10.195.10.100	10.195.10.1	TCP	66	[TCP Retransmission]
963	30.148595	10.195.10.100	10.195.10.1	DNS	90	Standard query 0x0d0
964	30.178314	10.195.10.1	10.195.10.100	TCP	54	53 → 52553 [RST, ACK]
965	30.178314	10.195.10.1	10.195.10.100	TCP	54	53 → 24778 [RST, ACK]
966	30.178314	10.195.10.1	10.195.10.100	TCP	54	53 → 25436 [RST, ACK]
967	30.178314	10.195.10.1	10.195.10.100	TCP	54	53 → 33658 [RST, ACK]
968	30.178314	10.195.10.1	10.195.10.100	ICMP	70	Destination unreachable
969	30.678700	10.195.10.100	10.195.10.1	TCP	66	[TCP Retransmission]
970	30.678813	10.195.10.100	10.195.10.1	TCP	66	[TCP Retransmission]
971	30.737943	10.195.10.1	10.195.10.100	TCP	54	53 → 25436 [RST, ACK]
972	30.737943	10.195.10.1	10.195.10.100	TCP	54	53 → 33658 [RST, ACK]
973	30.742408	10.195.10.100	10.195.10.1	DNS	86	Standard query 0x16d
974	30.742665	10.195.10.100	10.195.10.1	DNS	86	Standard query 0xab
975	30.751766	10.195.10.1	10.195.10.100	ICMP	70	Destination unreachable
976	30.751766	10.195.10.1	10.195.10.100	ICMP	70	Destination unreachable
977	31.552571	10.195.10.100	10.195.10.1	DNS	78	Standard query 0x2b

> Frame 1: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0	0000	4a 3f da 64 c3 0a 30 03	c8 26 5a fd 08 00 45
> Ethernet II, Src: CloudNet_26:5a:fd (30:03:c8:26:5a:fd), Dst: 08:00:00:00:00:00	0010	00 34 07 13 40 00 80 06	c9 c6 0a c3 0a 64 0a
> Internet Protocol Version 4, Src: 10.195.10.100, Dst: 10.195.10.1	0020	0a 01 38 a1 00 35 f1 d0	88 9a 00 00 00 00 80
> Transmission Control Protocol, Src Port: 14497, Dst Port: 52553	0030	ff ff 91 e4 00 00 02 04	05 b4 01 03 03 08 01
	0040	04 02	

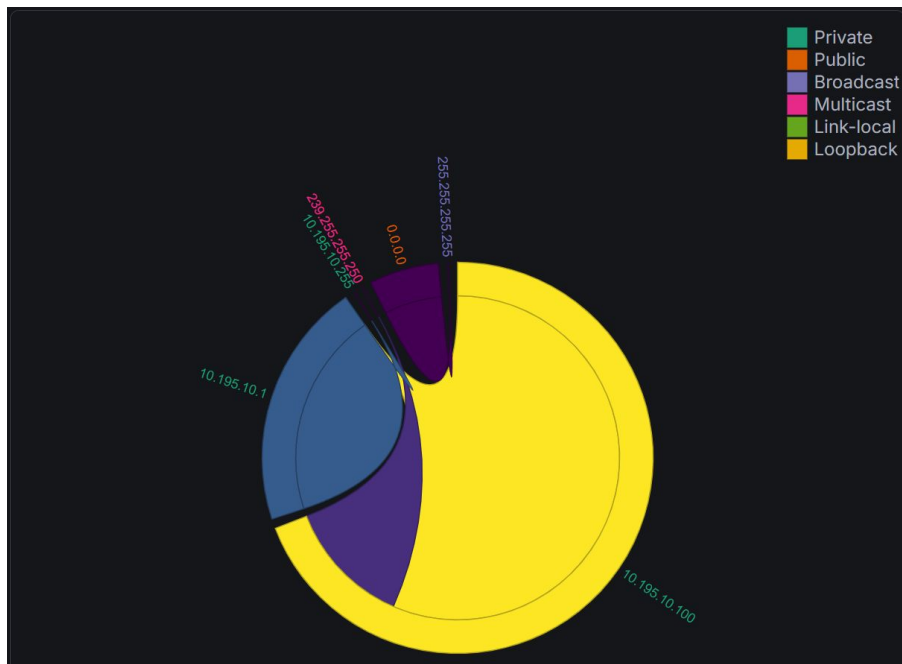


```
#define MESH_PREFIX "whateverYouLike"
#define MESH_PASSWORD "somethingSneaky"
```

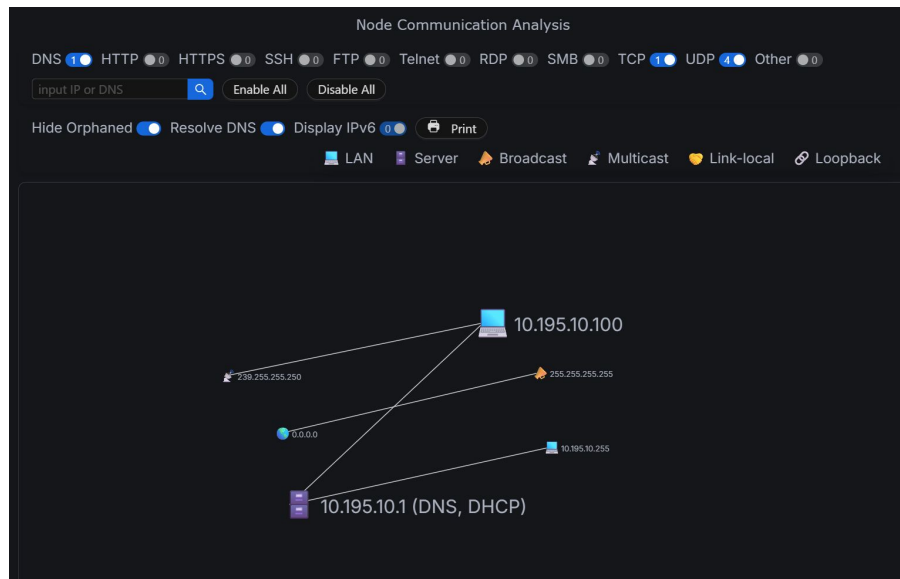
Analyzing pcapng

Question to look into:
What are the ESPs IP?

- <https://apackets.com/pcaps/ip> Is good for visualization
- Connection distribution:

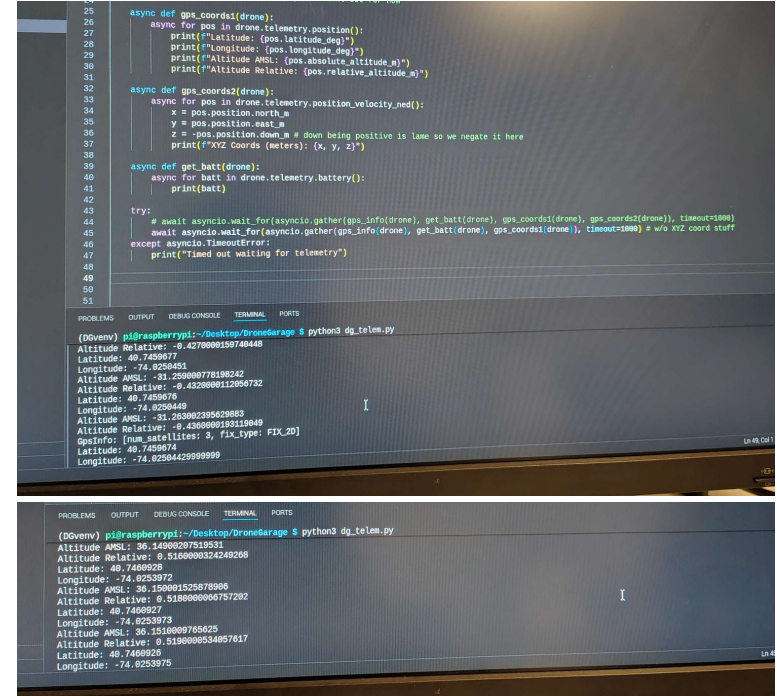


- Comm nodes:



GPS Data

- Succeeded in retrieving longitude, latitude, and altitude metrics
- Depicted in the images is a script running on the Pi that retrieves and displays such information, highlighting a change in the longitude value (in the thousandths), meaning that the GPS data is live and accurate (also accurate when compared to the latitude and longitude values at the lab)



The image shows two screenshots of a terminal window on a Raspberry Pi. The top screenshot displays the Python script being executed, which uses the DroneKit library to retrieve and print drone telemetry data. The bottom screenshot shows the output of the script, which prints various metrics including altitude, latitude, longitude, and GPS information.

```
25 async def gps_coords1(drone):
26     async for pos in drone.telemetry.position():
27         print(f"Latitude: {pos.latitude_deg}")
28         print(f"Longitude: {pos.longitude_deg}")
29         print(f"Altitude AMSL: {pos.absolute_altitude_m}")
30         print(f"Altitude Relative: {pos.relative_altitude_m}")
31
32 async def gps_coords2(drone):
33     async for pos in drone.telemetry.position_velocity_ned():
34         x = pos.position.north_m
35         y = pos.position.east_m
36         z = -pos.position.down_m # down being positive is lame so we negate it here
37         print(f"XYZ Coords (meters): {x, y, z}")
38
39 async def get_batt(drone):
40     async for batt in drone.telemetry.battery():
41         print(batt)
42
43 try:
44     # await asyncio.wait_for(asyncio.gather(gps_info(drone), get_batt(drone), gps_coords1(drone), gps_coords2(drone)), timeout=1000)
45     await asyncio.wait_for(asyncio.gather(gps_info(drone), get_batt(drone), gps_coords1(drone)), timeout=1000) # w/o XYZ coord stuff
46 except asyncio.TimeoutError:
47     print("Timed out waiting for telemetry")
48
49
50
51
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(D0venv) pi@raspberrypi:~/Desktop/DroneSage \$ python3 dg_telem.py

```
Altitude Relative: -0.427000150740648
Latitude: 48.7409677
Longitude: -74.0250453
Altitude AMSL: -31.250000778198242
Altitude Relative: -0.432000012056732
Latitude: 48.7409678
Longitude: -74.0250449
Altitude AMSL: -31.263002395030083
Altitude Relative: -0.4300000193119049
GpsInfo: [num_satellites: 3, fix_type: FIX_2D]
Latitude: 48.7409674
Longitude: -74.025044229999999
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

(D0venv) pi@raspberrypi:~/Desktop/DroneSage \$ python3 dg_telem.py

```
Altitude AMSL: 36.1490020751931
Altitude Relative: 0.5100000324248268
Latitude: 48.7409628
Longitude: -74.0253972
Altitude AMSL: 36.150001525078096
Altitude Relative: 0.51800000606757282
Latitude: 48.7409927
Longitude: -74.0253973
Altitude AMSL: 36.1510000765625
Altitude Relative: 0.5190000534057617
Latitude: 48.7409926
Longitude: -74.0253975
```